

Ninth Edition

Be Prepared
for the

AP

Computer
Science
Exam in Java

Maria Litvin
Gary Litvin

Skylight Publishing
Andover, Massachusetts

**Copyright © 2026 by
Maria Litvin, Gary Litvin, and Skylight Publishing**

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, or transmitted, in any form or by any means, electronic, mechanical, photocopying, recording, or otherwise, without the prior written permission of the authors and Skylight Publishing.

ISBN 978-0-9972528-3-5

Skylight Publishing
9 Bartlet Street, Suite 70
Andover, MA 01810

web: www.skylit.com
email: sales@skylit.com
support@skylit.com

1 2 3 4 5 6 7 30 29 28 27 26

Printed in the United States of America

Brief Contents

Preface vii

How to Use This Book x

Chapter 1. Exam Format, Grading, and Tips 1

Chapter 2. Java Features, Part 1 13

Chapter 3. Java Features, Part 2 55

Chapter 4. Algorithms 81

Chapter 5. Annotated Solutions to Past Free-Response Questions ☆
www.skylit.com/beprepared

Practice Exams 101

Answers and Solutions 229

Index 267

About the Authors

Maria Litvin taught computer science and mathematics at Phillips Academy in Andover, Massachusetts for over 30 years. She has been an AP Computer Science exam reader and Table and Question Leader, and as a consultant for the College Board, provides AP training for high school computer science teachers. Maria is a recipient of the 1999 Siemens Award for Advanced Placement for Mathematics, Science, and Technology for New England, the 2003 RadioShack National Teacher Award, and the 2020 NCWIT Aspirations in Computing Educator Award. Prior to joining Phillips Academy, Maria taught computer science at Boston University.

Maria co-authored the first, (C++) version of *Be Prepared for the AP Computer Science Exam* (Skylight Publishing, 1999) and eight previous Java editions. She is also a co-author, with Gary Litvin, of several popular computer science textbooks: *C++ for You++: An Introduction to Programming and Computer Science*, which was the leading high school textbook for AP Computer Science courses in the C++ era, *Java Methods: Object-Oriented Programming and Data Structures*, now in its fourth AP Edition, *Coding in Python and Elements of Discrete Mathematics* (Skylight Publishing, 2019), and *Bits and Chips: Computer Science in Questions and Puzzles for Aspiring Coders*, second edition (Skylight Publishing, 2022).

Gary Litvin is a co-author, with Maria Litvin, of *C++ for You++*, the *Java Methods* series, the fourth and later editions of *Be Prepared for the AP Computer Science Exam in Java*, *Coding in Python*, and *Bits and Chips*. Gary has worked in many areas of software development including artificial intelligence, pattern recognition, computer graphics, and neural networks. As the founder of Skylight Software, Inc., he developed SKYLIGHTS/GX, one of the first visual programming tools for C and C++ programmers. Gary led the development of several state-of-the-art software products, including interactive touch screen development tools, OCR and handwritten character recognition systems, and credit card fraud detection software.

Contents

Preface vii

How to Use This Book x

Chapter 1. Exam Format, Grading, and Tips **1**

- 1.1. Exam Format and Materials 1
- 1.2. The AP Java Subset 3
- 1.3. Tested Terms, Concepts, and Algorithms 5
- 1.4. Grading 6
- 1.5. College Credit 8
- 1.6. Exam-Taking Tips 9

Chapter 2. Java Features, Part 1 **13**

- 2.1. Variables; Arithmetic, Relational, and Logical Operators 13
- 2.2. Conditional Statements and Loops 22
- 2.3. Strings 31
- 2.4. Integer and Double Classes 35
- 2.5. Arrays 37
- 2.6. The ArrayList Class 44
- 2.7. Exceptions 47
- 2.8. `print` and `println` 49
- 2.9. Reading Text Files 51

Chapter 3. Java Features, Part 2 **55**

- 3.1. Classes 55
- 3.2. Static Variables and Methods 61
- 3.3. Calling Methods 64
- 3.4. Random Numbers 75
- 3.5. Class Libraries and the API 76
- 3.6. Program Design and Development Methodology 77
- 3.7. The “Class” Question 78
- 3.8. Inheritance 79

Chapter 4. Algorithms **81**

- 4.1. Iterations 81
- 4.2. Sequential Search and Binary Search 88
- 4.3. Selection Sort and Insertion Sort 91
- 4.4. Recursion 93
- 4.5. Mergesort 98

Chapter 5. Annotated Solutions to Past Free-Response Questions



www.skylit.com/beprepared

Practice Exams 101

- Exam #1 103
- Exam #2 137
- Exam #3 169
- Exam #4 199

Answers and Solutions 229

- Exam #1 229
- Exam #2 237
- Exam #3 247
- Exam #4 255

Index 267

Index to Free-Response Questions 273

Preface

The AP Computer Science A (AP CSA) exam tests your understanding of basic concepts in computer science as well as your familiarity with Java programming. The exam covers roughly the material of a one-semester introductory college course in computer science (CS-1).

The designation “A” is a vestige of the earlier era (before 2010) when there were two exams, “A” and “AB”; the latter included data structures material: linked lists, stacks, queues, binary trees, and so on. More recently, the College Board added the AP Computer Science: Principles (AP CSP) course and exam. AP CSP is a lower-level exam that does not have a designated programming language and does not require coding fluency in any particular programming language.

In the spring of 2004, the computer science exams used Java for the first time.

Solutions to free-response exam questions written in a programming language other than Java or in pseudocode will not receive credit.

A working knowledge of Java is necessary but not sufficient for a good grade on the exam. You must also understand basic computer science concepts and be able to implement in Java some standard algorithms. As for Java: you don’t have to know the whole language, just a subset.

AP CSA Exam questions are written by the College Board’s AP Computer Science A Exam Development Committee, and exams are administered by the Educational Testing Service (ETS). The College Board currently offers 38 AP exams; in a recent year, nearly three million students took over five million AP Exams.

In the spring of 2025, the College Board redesigned the AP CSA exam and updated the *Computer Science A Course and Exam Description* (CED). The new CED, effective Fall 2025, has four units:

- Unit 1: Using Objects and Methods
- Unit 2: Selection and Iteration
- Unit 3: Class Creation
- Unit 4: Data Collections

Each unit specifies several “Learning Objectives”; each “Learning Objective” states in more detail several “Essential Knowledge” items.

The College Board has eliminated all object-oriented programming (OOP) topics from the exam. Inheritance, superclasses and subclasses, class hierarchies, and polymorphism are all excluded. What remains is writing short classes, instantiating objects, calling methods of user-defined and Java library classes, and using control structures (if-else, for and while loops). “Data collections” refers to one- and two-dimensional arrays and the `ArrayList` Java library class.

The format of the exam has changed, too: 42 multiple-choice questions instead of 40, each with four answer choices instead of five; shorter free-response questions. The exam’s four free-response questions test specific topics:

Question 1: Methods and Control Structures

Question 2: Write a small class

Question 3: Write one method that manipulates data in an `ArrayList`

Question 4: Write one method that uses a 2D Array

AP CSA exams are now fully digital, administered online using the College Board’s “Bluebook” testing app. During the exam, you will be given several sheets of scratch paper, and you may request more, if needed. While you need a computer with a Java compiler to learn how to program and how to implement common algorithms in Java, this book does not require the use of a computer. One-hundred-percent correct Java syntax is not the emphasis here; small mistakes (a missed semicolon or brace) that a compiler would normally help you catch will probably not affect your exam score. You’ll need a computer only to access collegeboard.org and our website, www.skylit.com/beprepared, for the latest updates and our solutions to the free-response questions from past exams. It is important to download and practice with Bluebook (bluebook.collegeboard.org/students/download-bluebook) before the exam.

It is not the goal of this book to teach you everything you need to know from scratch. For that, you need a complete textbook with exercises and coding projects. Most students who take the exam are enrolled in an AP CSA course at their school. A determined student can prepare for the exam on their own; it may take anywhere between one and ten months, and a good textbook will be even more important.

The goals of this book are to:

- describe the exam format and requirements
- describe the AP Java subset
- provide a thorough review of what you should know, with emphasis on the more difficult topics and common omissions and mistakes
- help you identify and fill the gaps in your knowledge
- offer sample exam questions with answers, hints, and solutions for you to practice with and analyze your mistakes

The College Board requires AP CSA courses to have a lab component — at least 20 hours of hands-on Java programming. Students are expected to use a computer for the course at least 3 hours per week. The Development Committee has made available six sample labs (only in the AP Classroom platform for teachers). These labs are samples only, for demonstrating the extent of lab work expected; they will not be tested on the AP exams.

Chapter 1 of this book explains the format and the Java subset for the exam and provides information about exam grading and exam-taking hints. Chapter 2 and Chapter 3 cover the elements of Java required for the exam. Chapter 4 reviews common searching and sorting algorithms and recursion. Chapters 1-4 contain sample multiple-choice questions with detailed explanations of all the right and wrong answers.

Chapter 5 is actually on the web at this book's companion website, <http://www.skylit.com/beprepared>. It offers our annotated solutions to the free-response questions from past exams. At the end of the book are four complete practice exams followed by answers and solutions.



We thank the teachers and students who alerted us to several mistakes in the earlier editions of this book.

Our special thanks to Margaret Litvin and Aaron Litvin for making this book more readable with their thorough and thoughtful editing.

How to Use This Book

The companion website

www.skylit.com/beprepared

is an integral part of the book. It contains annotated solutions to free-response questions from past exams and relevant links. Check this website for current information, and be sure you have the latest edition of *Be Prepared*.

Multiple-choice questions in the review chapters are marked by their number in a box:

1

Their solutions are delimited by  and .

Our practice exams may be slightly more difficult than the actual exams, so don't panic if they take more time.

The book's review chapters include several worked out multiple-choice questions with three options, marked I, II, and III.

Multiple-choice questions with options I, II, III, will not appear on the AP exams and are not included in our practice exams.

 Braces like these on the margins in the review chapters mark a comment about a feature not tested on the AP exams.

 In the free-response questions in our practice exams, such braces indicate suggestions for additional lab work based on our questions. Starter code for these labs is available in Student Files, downloadable from www.skylit.com/beprepared. Complete solutions to the labs are available to teachers; to get access, email support@skylit.com from your school email account and include a link to the school's web page that lists your email address.

Chapter 1 Exam Format, Grading, and Tips

1.1 Exam Format and Materials

Figure 1-1 shows the format of the AP Computer Science A exam. The exam takes three hours of test time, plus a break and time for instructions. The exam is divided into two sections. Section I consists of 42 multiple-choice questions with a total allotted time of 1 hour and 30 minutes (a little over two minutes per question, on average). Section II consists of four free-response questions with a total allotted time of 1 hour and 30 minutes. The first free-response question has two parts; the remaining three questions have one part each.

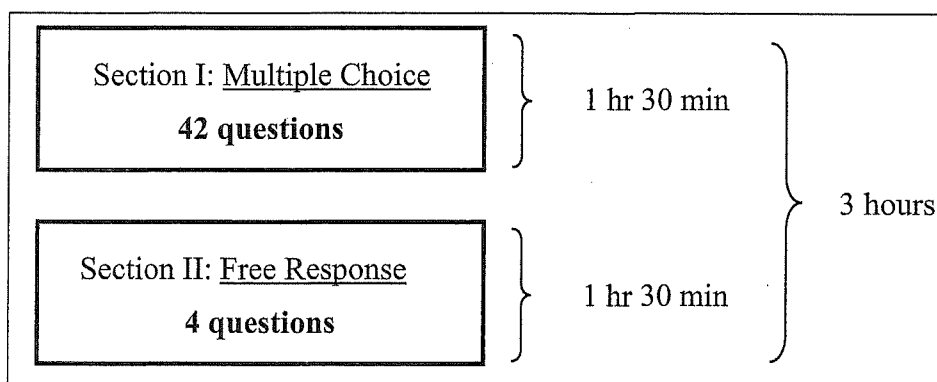


Figure 1-1. AP Computer Science A exam format

The exam is administered online via the Bluebook test-taking app; scratch paper is provided.

Bluebook will notify you five minutes before the time expires, both in the multiple-choice and free-response sections.

During the entire exam you will have access to the *Java Quick Reference*, which provides a list of the Java library classes and their methods included in the AP Java subset. Students and teachers can find a copy of the *Java Quick Reference* in the Course and Exam Description (CED). It is expected that you will already be very familiar and comfortable with the required Java library classes and their methods before the exam.

The multiple-choice section is a mixture of questions related to general computer science terms, program design decisions, specific elements of Java syntax, analysis of fragments of Java code, basic properties of a class, creating objects and accessing their methods. The multiple-choice section is graded out of 42 points with all questions weighted equally and no penalty for wrong answers. It accounts for 55% of the total exam score. The free-response section is graded out of 25 points and accounts for 45% of the total exam score.

The topics of the four free-response questions test specific skills:

Question 1: “Methods and Control Structures” (7 points total)

Part (a): Write a constructor or a method that may involve `if-else` and loops (4 points)

Part (b): Call `String` methods; you may need to call the method from Part (a) (3 points)

Question 2: “Class Design” (7 points)

Write a short class with instance variables, a constructor, and one method

Question 3: “Data Analysis with `ArrayList`” (5 points)

Write one method that examines and/or manipulates the elements of an `ArrayList`

Question 4: “2D Array” (6 points)

Write one method that examines and/or manipulates the elements of a 2D array

Part (b) of Question 1 may refer to the method implemented in Part (a). Each part is graded separately, and your implementation of Part (a) does not have to be correct in order for you to receive full credit for Part (b).

Your implementation of the class in Question 2 will be graded based on the correctness of the implementation of your class, including the *encapsulation* principle (declaring all instance variables `private`), and other criteria.

The College Board requires a typical AP CSA course to include at least 20 hours of computer programming lab work. The Development Committee has made available six sample labs for practice (available exclusively in the College Board’s AP Classroom).

These labs are only examples designed to illustrate the scope and difficulty of lab work in a typical AP CSA course. The content and code in these labs will not be tested on the AP CSA exams.

1.2 The AP Java Subset

There is no formal definition of the AP Java subset in the CED, but you can glean the elements of the subset from the *Essential Knowledge* components and the “Exclusion Statements” in the CED.

So what’s in the subset? Actually, quite a bit:

- Comments: `/* ... */`, `//`, and `/** ... */`; preconditions and postconditions.
- `boolean`, `int`, and `double` primitive data types. `(int)` and `(double)` casts. **Other primitive data types, including `char`, are not in the subset and should be avoided on the exam.**
- `Integer.MIN_VALUE` and `Integer.MAX_VALUE`
- The assignment operator `=`. The arithmetic operators `+`, `-`, `*`, `/`, `%`. The increment/decrement operators `++` and `--` (**use only the postfix form `k++` or `k--`, and do not use them in expressions**). The compound assignment operators `+=`, `-=`, `*=`, `/=`, `%=`. The relational operators `<`, `>`, `<=`, `>=`, `==`, `!=`. The logical operators `&&`, `||`, `!`.
- `if-else` statements. The keywords `true` and `false`.
- Literal strings in double quotes. `\n`, `\\`, and `\"` escape characters in literal strings. The `+` and `+=` operators for concatenating strings. `String`’s `length`, `substring` methods, `indexOf(String s)`, `compareTo`, `equals`, and `split` methods.
- `Integer.parseInt(String s)` and `Double.parseDouble(String s)` methods.
- `System.out.print` and `System.out.println`.
- `File(String str)` constructor. `Scanner(File f)` constructor. `IOException`. `Scanner`’s `nextInt`, `nextDouble`, `nextBoolean`, `nextLine`, `next`, `hasNext`, and `close` methods for reading a text file.
- One-dimensional and two-dimensional (**rectangular only**) arrays. `array.length`. Arrays of objects. Initialized arrays such as
`int[] arr = {1, 2, 3};`
`int[][] m = {{1, 2, 3}, {4, 5, 6}};`
Default values of elements for arrays defined using the `new` operator.

- for and while loops. The “enhanced” for loop (for-each loop):
`for (type x : values) ...`
- Classes. Instance variables. Constructors. The new operator. public and private methods. static methods (may be public). Overloaded methods. The return statement. void methods.
- static variables and static final variables (constants). null. this reference. **All instance variables are private.**
- Understanding `ArithmeticException`, `NullPointerException`, `IndexOutOfBoundsException`, `ArrayIndexOutOfBoundsException`, `ConcurrentModificationException`, `InputMismatchException`, and `IOException`.
- The `ArrayList<E>` class (see Section 2.6).
- File and Scanner classes (see Section 2.9).
- Other library classes, methods, and constants:
 - String:** `length()`, `substring(int from)`,
`substring(int from, int to)`, `indexOf(String s)`,
`equals`, `compareTo`, and `split`
 - Integer:** `Integer.parseInt()`; `Integer.MIN_VALUE` and
`Integer.MAX_VALUE`
 - Double:** `Double.parseDouble()`
 - Math:** `abs(int x)`, `abs(double x)`,
`pow(double base, double exp)`, `sqrt(double x)`,
`random()`
 - Object:** `equals(Object other)`, `toString()`

Also understand the `toString` and `equals` methods for all objects and the `compareTo` method for the `String`, `Integer`, and `Double` objects.

If you feel you must stray from this subset in your free-response solution, you might have misunderstood the problem and be making it harder than it is.

At the same time, it is okay to use such out-of-the-subset features as `Math.min(x, y)` and `Math.max(x, y)` methods, `ArrayList`’s `contains` and `remove(object)` methods, `break` in loops, and other simple tools that all exam readers are familiar with.

Things that are not in the AP Java subset and should be avoided include the following:

- Java syntax abominations, such as the `?_:_` operator
- `++` and `--` in expressions (as in `a[i++]`) and the prefix form of the `++` and `--` operators (`++k`, `--k`)
- Primitive data types other than `int`, `double`, and `boolean` (`char`, `long`, and `float` are not in the subset)
- Bitwise logical operators and shift operators: `~`, `&`, `|`, `^`, `<<`, `>>`.
- The `switch` statement, the `do-while` loop, `break` and `continue` in loops
- Library classes and methods other than those mentioned in *Java Quick Reference*
- Abstract classes and interfaces
- Checked exceptions and `try-catch-finally` statements
- Any input other than `File`, `Scanner`, and `IOException` classes which are used only for reading from a text file. (Reading values from the keyboard using `Scanner` class is excluded).
- Any output other than `System.out.print` and `System.out.println`
- `enum` data types
- Subclasses and interfaces, `extends`, `implements`, `super`, method overriding

1.3 Tested Terms, Concepts, and Algorithms

In addition to Java, you need to be familiar with the following terms, concepts, and algorithms mentioned in the *Essential Knowledge* components of the CED:

- Rounding to the nearest integer: `(int) (x + 0.5)` for a positive double `x`; `(int) (x - 0.5)` for a negative `x`
- *Autoboxing / unboxing*
- Constructor and method *signature* (the method's name and the list of data types of parameters)
- *Invoking* constructors using `new`; calling methods using the “dot” operator
- *Local* variables; *instance variables* and *class (static) variables*
- Constructor and method *parameters*
- Passing arguments *by value* (an object is passed as a copy of the reference)
- *Accessor* and *mutator (modifier)* methods

- Array / ArrayList algorithms: traverse, find the minimum or maximum, find the sum or average of elements, examine all pairs of consecutive elements, find duplicates, examine all elements for a given property, shift or rotate elements left or right
- Sequential Search and iterative and recursive versions of Binary Search
- Selection Sort, Insertion Sort, and Mergesort
- Understand (**but not write**) recursive code

The Java subset's features and the above terms, concepts, and algorithms are reviewed in the subsequent chapters.

1.4 Grading

The exam is graded on a scale from 1 to 5. Table 1-1 shows the College Board's college credit recommendations and grade equivalents for AP grades.

AP Grade	Credit Recommendation	College Grade Equivalent
5	Extremely well qualified	A
4	Well qualified	A-, B+, B
3	Qualified	B-, C+, C
2	Possibly qualified	n/a
1	No recommendation	n/a

Table 1-1. AP grades, credit recommendations, and college grade equivalents

Grades of 5 and 4 are called “extremely well qualified” and “well qualified,” respectively, and usually will be honored by colleges that give credit or placement for AP CSA. A grade of 3, “qualified,” may be denied credit or placement at some colleges. Grades of 2 (“possibly qualified”) and 1 (“no recommendation”) will not result in college credit or placement.

Table 1-2 presents published statistics and grade distributions for the 2024 and 2023 exams. 98,136 students took the exam in 2024; 47.0 percent of them scored 4 or 5.

AP Computer Science A	2024		2023	
	Number	%	Number	%
Students	98,136	100.0	94,438	100.0
Grade:				
5	25,137	25.6	25,269	26.8
4	21,038	21.4	21,181	22.4
3	19,754	20.1	17,730	18.8
2	10,613	10.8	8,984	9.5
1	21,594	22.0	21,274	22.5
4 or 5	46,175	47.0	46,450	49.2

Table 1-2. 2024 and 2023 grade distributions

The multiple-choice section counts for 55% of the final grade, while the free-response section counts for 45%.

The College Board uses a weighted combination of the multiple-choice (MC) and free-response (FR) scores to determine the final total score:

$$\text{totalScore} = \text{MC_coeff} * \text{countCorrect} + \text{FR_coeff} * \text{FR_score};$$

One point is given for each correct answer to a multiple-choice question.

There is no penalty for giving a wrong answer to a multiple-choice question, so it is a good strategy not to leave any answers blank. Cross out the choices that appear wrong and guess among the remaining ones. (Use Bluebook's cross-out feature.)

Solutions to free-response questions are graded by a group of high school teachers and college professors. Scores are based on a *rubric* established by the Chief Reader, with input from the Exam Leader, Question Leaders, and Table Leaders. The total possible number of points is 25, with 7, 7, 5, and 6 points for Questions 1-4, respectively.

The final score is obtained by adding the MC and FR weighted scores. The MC and FR coefficients are chosen in such a way that they give 55% and 45% weights to the multiple-choice and free-response sections of the exam, respectively.

The cut-off points are determined by the Chief Reader in consultation with the College Board and may vary slightly from year to year based on the score distributions and close examination of a sample of individual exams. Table 1-3 shows the cut-off points from one of the past exams.

Percent range	Grade
77.5 - 100	5
55.0 - 77.4	4
38.8 - 54.9	3
31.3 - 38.7	2
0 - 31.2	1

Table 1-3. Sample score cut-off points for grades

The College Board releases the free-response questions 48 hours after the exam and posts them on its website. We post our annotated solutions at www.skylit.com/beprepared a few hours after the questions are posted. The College Board posts the scoring rubrics for the free-response questions sometime in the summer. Every few years the College Board releases a complete exam, including a diagnostic guide for the multiple-choice questions, scoring rubrics for the free-response questions, and the cut-off points for grades. See www.skylit.com/beprepared/notes.txt for the latest updates.

1.5 College Credit

Most colleges will take your AP courses taken and exam grades into account in admission decisions. But acceptance of AP exam results for credit and/or placement varies widely among colleges. In general, the AP CSA course corresponds to a CS-1 course (Introductory Computer Science or Computer Programming I), a one-semester course for computer science majors. Some colleges base their decision on your grade, and some may not give any credit at all. Consult the websites of the colleges you are interested in and the College Board's AP credit policies page, <https://apstudent.collegeboard.org/creditandplacement/search-credit-policies>.

1.6 Exam-Taking Tips

Some things are obvious:

- If you took the time to read a multiple-choice question and all the answer choices, take an extra ten seconds and guess. Most likely you have eliminated one or two wrong answers even without noticing. **Do not leave any multiple-choice answers blank; there is no penalty for wrong answers.**
- Do read the question before jumping to the code included in the question. Notes to multiple-choice questions in our practice exams might teach you some shortcuts.

There are a few important things to know about answering free-response questions.

In a nutshell: be neat, straightforward, and professional; keep your exam reader in mind, and don't show off.

More specifically:

1. Stay within the AP Java subset, except for a few obvious shortcuts, such as `Math.max(...)` and `Math.min(...)`.
2. Remember that the elegance of your code does not count. More often than not, a brute-force approach is the best. If you write tricky, non-standard code, you may waste a lot of time and trick yourself in the process, or mislead your exam reader, who, after all, is only human. Your exam reader will read your solution but will not test it on a computer.
3. The efficiency of your code does not count, unless the desired performance of the solution is specifically stated in the question, which is unlikely.
4. Remember that Part (b) of Question 1 is graded independently from Part (a) and may actually be easier. It is not uncommon for the method specified in Part (a) to be called in Part (b). Do so, even if your Part (a) is incorrect or left blank. Do not re-implement code from Part (a) in Part (b) — you will waste valuable time and may lose points for doing so.
5. If a question presents a partial definition of a class with certain methods described but not implemented (“implementation not shown”), call these methods whenever appropriate in your code — do not write equivalent code yourself.

6. Bits of “good thinking” count. You may not know the whole solution, but if you have read and understood the question, go ahead and write fragments of code that may earn you partial credit. On the free-response Question 2, write the class header, constructor, and method headers even if you are not sure how to implement them. But don’t spend too much time improvising incorrect code.
7. You cannot submit two attempts at a solution in Bluebook. If you are unsure about your solution, copy and paste it at the top of the solution box, then review and edit as necessary. Once you are happier with the result, delete the previous version below. If you believe this attempt didn’t improve things, delete the copy at the top.
8. Read the comment above the method header quickly — it usually restates the task in a more formal way and sometimes gives valuable hints. Assume that all preconditions are satisfied — do not check them in your code!
9. One common mistake is to forget a `return` statement in a non-void method. Make sure the returned value matches the specified method return type.
10. Do not ignore hints in the question description. If an algorithm is suggested for a method (as in “you may use the following algorithm”), don’t fight it, just use it!
11. Remember that the exam readers score a vast number of exams in quick succession during a marathon grading session every June. Space out your code. Always indent your code properly. This helps you and your exam reader. If you miss a brace but your code is properly indented, the reader (as opposed to a Java compiler) may accept it as correct. Similarly, if you put each statement on a separate line, a forgotten semicolon might not be held against you.
12. Follow the Java naming convention: the names of all methods, variables, and parameters start with a lowercase letter. Use meaningful, but not too verbose, names for variables. `count` may be better than `a`; `sum` may be better than `temp`; `row`, `col` or `r`, `c` may be better than `i`, `j`. But `k` is better than `loopControlVariable`. If the question contains examples of code with names, use the same names when appropriate.

13. Don't bother with comments; they do not count, and you will lose valuable time. Occasionally you can write a very brief comment that indicates your intentions for the fragment of code that follows — but only if it helps you. For example:

```
// Find the first empty seat:  
...  
...
```

14. Don't worry about `imports` — assume that all the necessary Java library classes are imported.
15. Code strictly according to the specifications, preconditions, and postconditions. Avoid extraneous “bells and whistles” — you will lose points. Never add `System.out.print/println` in solutions unless specifically asked to do so. Do not read any data in your method — it is passed in as arguments or is available as instance or static variables of the class.
16. Do not use in your code specific numbers, strings, or dimensions of arrays given as examples in explanations of questions. If the question says, “For example, a two-dimensional array `pixelValues` may contain the following image” and shows an array of 4 rows by 5 columns, do not use 4 and 5 in your code — make your code work with an array of any size.
17. In Question 3, remember that you are working with an `ArrayList` and use its `size`, `get`, and `set` methods, not array's `length` and brackets for accessing elements.
18. In Question 4, given a 2D array `arr`, `arr.length` gives the number of rows and `arr[0].length` gives the number of columns. The first index is the row and the second index is the column, as in `arr[r][c]`.
19. Don't try to catch the exam authors on ambiguities: there will be no one to hear your case, and you'll waste your time. Instead, try to grasp quickly what was meant and write your solution.
20. Don't quit until the time is up. Use all the time you have and keep trying. The test will be over before you know it.

Chapter 2 Java Features, Part 1

2.1 Variables; Arithmetic, Relational, and Logical Operators

Primitive data types included in the subset are `boolean`, `int`, and `double`. In Java, an `int` always takes four bytes, regardless of a particular computer or Java compiler, and its range is from -2^{31} to $2^{31} - 1$. The smallest and the largest integer values are defined in Java as symbolic constants `Integer.MIN_VALUE` and `Integer.MAX_VALUE`; use these symbolic constants if you need to refer to the limits of the `int` range. A `double` takes 8 bytes and has a huge range, but its precision is about 15 significant digits.

 **Remember to declare local variables.**

If you declare a variable inside a nested block, make sure it is used only in that block. If you declare a variable in a `for` loop, it will be undefined outside that loop.

For example:

```
public int countMins(int[] a)
{
    if (a.length > 0)
    {
        int iMin = 0;

        for (int i = 1; i < a.length; i++)
        {
            if (a[i] < a[iMin])
                iMin = i;
        }

        int count = 0;
        for (i = 0; i < a.length; i++)
        {
            if (a[i] == a[iMin])
                count++;
        }
    }
    return count;
}
```

Error: i is undefined



Error: count is undefined here



A safer version:

```
public int countMins(int[] a)
{
    int iMin = 0;
    int count = 0;

    if (a.length > 0)
    {
        iMin = 0;
        for (int i = 1; i < a.length; i++)
        {
            if (a[i] < a[iMin])
                iMin = i;
        }

        count = 0;

        for (int i = 0; i < a.length; i++)
        {
            if (a[i] == a[iMin])
                count++;
        }
    }
    return count;
}
```

Do not declare count here —
int count = 0;
would be a mistake

You won't be penalized for declarations inside blocks of code, but if you declare important variables near the top of the method's code, it makes it easier to read and may help you avoid mistakes.

1

Which of the following statements is true?

- (A) In Java, data types in declarations of symbolic constants are needed only for documentation purposes.
- (B) When a Java interpreter is running, variables of the `double` data type are represented in memory as strings of decimal digits with a decimal point and an optional sign.
- (C) A variable's data type may determine whether that variable may be passed as an argument to a particular method.
- (D) A variable of the `int` type cannot serve as an operand for the `/` operator.

 This question gives us a chance to review what we know about data types. Choice A is false: the data type of a symbolic constant is used by the compiler. In this sense, symbolic constants are not so different from variables. The difference is that a constant declaration includes the keyword `final`. Class constants are often declared as `static final` variables. For example:

```
public static final double LBS_IN_KG = 2.20462262;
public static final int maxNumSeats = 120;
```

Choice B is false, too. While real numbers may be written in programs in decimal notation, a Java compiler converts them into a special “double” floating-point format that takes eight bytes and is convenient for computations.

Choice D is false, too. In Java, you can write `a/b`, where both `a` and `b` are of type `int`. The result is truncated to an integer.

Choice C is true. For example, a value of the type `String` cannot be passed to a method that expects an `int` as a parameter. Sometimes, though, an argument of a different type may be promoted to the type expected by the method — for example, an `int` can be promoted to a `double` when you call, say, `Math.sqrt(x)` for an `int x`. The answer is C. 

~~~~~ Arithmetic operators ~~~~~

 The most important thing to remember about Java’s arithmetic operators is that the data type of the result, even each intermediate result, is the same as the data type of the operands. In particular, the result of division of one integer by another integer is truncated to an integer.

For example:

```
int n = 3;
double result;

result = (n + 1) * n / 2;           // result is 6.0
result = (n / 2) * (n + 1);        // result is 4.0
result = (1 / 2) * n * (n + 1);    // result is 0.0
```

To avoid truncation, you have to watch the data types and sometimes use the *cast operator*. For example:

```
int a, b;
double ratio;
...
ratio = (double)a / b;           // Or: a / (double)b;
// But not ratio = (double)(a/b) -- this is a cast applied too late!
```

If at least one of the operands is a double, there is no need to cast the other one — it is promoted to a double automatically. For example:

```
int factor = 3;
double x = 2.0 / factor; // Result: x = 0.6666...
```

2

Which of the following expressions does not evaluate to 0.4?

- (A) `(int)4.5 / (double)10;`
- (B) `(double)(4 / 10);`
- (C) `4.0 / 10;`
- (D) `(double)4 / (double)10;`

 In Choice B the cast to double is applied too late — after the ratio is truncated to 0 — so it evaluates to 0. The answer is B. 

In the real world, we have to worry about the range of values for different data types. For example, a method that calculates 3^n as an `int` may overflow the result, even for a relatively small n . In Java, arithmetic overflow is not reported in any way: there is no warning or exception. If an `int` result is greater than or equal to 2^{31} , the leftmost bit, which is the sign bit, may be set to 1, in which case the result would become negative.

For the AP exam, you have to be aware of what overflow is, and you need to be aware of the `Integer.MIN_VALUE` and `Integer.MAX_VALUE` constants, but you don't have to know their specific values.

Modulo division

The `%` (modulo division) operator usually applies to two integers: it calculates the remainder when the first operand is divided by the second.

For example:

```
int r;
r = 17 % 3; // r is set to 2
r = 8 % 2;  // r is set to 0
r = 4 % 5;  // r is set to 4
```

On the AP exam, a `% b` will only be used with $a \geq 0$ and $b > 0$.

~~~~~ *Compound assignments and ++ and --* ~~~~~

The compound assignment operators are `+=`, `-=`, `*=`, `/=`, and `%=`. `x += y` is the same as `x = x + y`. Other operations follow the same pattern.

There are two forms of the `++` and `--` operators in Java. The prefix form ("`++k`") increments (or decrements) the variable before its value is used in the rest of the expression, whereas the postfix form increments (or decrements) it afterwards.

The AP CS Development Committee discourages the use of `++` and `--` in expressions. Use `++` and `--` only in separate statements and only in the postfix form, such as `k++` or `k--`.

For example:

```
while (i <= n)
{
    sum += i++;
}
```

Too much!

```
while (i <= n)
{
    sum += i;
    i++;
}
```

OK

You won't lose points over `++` or `--` in expressions if you use them correctly, but they won't earn you any extra credit, either.

It is bad style not to use increment or compound assignment operators where appropriate.

For example:

```
for (int i = 0; i < n; i = i + 1)
{
    count = count + 1;
    sum = sum + a[i];
}
```

Works, but looks bad

```
for (int i = 0; i < n; i++)
{
    count++;
    sum += a[i];
}
```

Better

Again, this incurs no penalty but doesn't look good.

Arithmetic expressions are too easy to be tested alone. You may encounter them in questions that combine them with logic, iterations, recursion, and so on.

Relational operators

In the AP Java subset, the relational operators `==`, `!=`, `<`, `>`, `<=`, `>=` apply to `ints` and `doubles`. Remember that “is equal to” is represented by `==` (not to be confused with `=`, the assignment operator). `a != b` is equivalent to `!(a == b)`, but `!=` is stylistically better.

The `==` and `!=` operators can also be applied to two objects, but their meanings are different from what you might expect: they compare the addresses of the objects. The result of `==` is true if and only if the two variables refer to exactly the same object. You rarely care about that: most likely you want to compare the contents of two objects, for instance, two strings. Then you need to use the `equals` or `compareTo` method.

For example:

```
if (str.equals("Stop")) ...
```

On the other hand, `==` or `!=` must be used when you compare an object to `null`.

`null` is a Java reserved word that stands for a reference with a zero value. It is used to indicate that a variable currently does not refer to any valid object. For example:

```
if (str != null && str.equals("Stop")) ...  
// str != null avoids NullPointerException --  
// can't call a null's method
```

You can instead write:

```
if ("Stop".equals(str)) ...
```

This works because `"Stop"` is not `null`; it works even if `str` is `null`.

You won't be asked to write an `equals` method in free-response Question 2.

So, never do that — you will waste your time and won't earn any extra points. In fact, you may get a deduction if you make a mistake.

Logical operators

The logical operators `&&`, `||`, and `!` normally apply to Boolean values and expressions. For example:

```
boolean found = false;
...
while (i >= 0 && !found)
{
    ...
}
```

```
boolean found = false;
...
while (i >= 0 && found == false)
{
    ...
}
```

Works, but is more verbose

Do not write

```
while (... && !found == true)
```

— this works, but “`== true`” is redundant.

Two logical expressions are equivalent if they have equal values for all possible true/false combinations of values of variables in them.

3

Assuming that `x`, `y`, and `z` are declared and initialized integer variables, which of the following three logical expressions are equivalent to each other — that is, have equal values for all possible values of `x`, `y`, and `z`?

I. `(x == y && x != z) || (x != y && x == z)`

II. `(x == y || x == z) && (x != y || x != z)`

III. `(x == y) != (x == z)`

- (A) Only I and II
- (B) Only II and III
- (C) Only I and III
- (D) All three

 Expression III is the key to the answer: all three expressions state that exactly one out of the two equalities, `x == y` or `x == z`, is true. Expression I states that either the first and not the second or the second and not the first is true. Expression II states that one of the two is true and one of the two is false. Expression III simply states that they have different values. All three boil down to the same thing. The answer is D. 

~~~~~ De Morgan's Laws ~~~~~

The exam is likely to include questions on De Morgan's Laws:

`!(a && b)` is the same as `!a || !b`
`!(a || b)` is the same as `!a && !b`

4

The expression `!((x <= y) && (y > 5))` is equivalent to which of the following?

- (A) `(x <= y) && (y > 5)`
- (B) `(x <= y) || (y > 5)`
- (C) `(x > y) || (y <= 5)`
- (D) `(x > y) && (y <= 5)`

 The given expression is pretty long, so if you try to plug in specific numbers you may lose a lot of time. Use the De Morgan's Laws instead:

`!((x <= y) && (y > 5))`
`!(x <= y) || !(y > 5)`
`(x > y) || (y <= 5)`

When `!` is distributed,
`&&` changes into `||`
 and vice versa

The answer is C. 

It is possible to establish that two expressions are equivalent by comparing their truth tables. A truth table is a table that lists the values of the expression for all combinations of values of variables in it.

It is not very practical to use truth tables for solving a multiple-choice question because creating them is time-consuming. It is more likely that the solution is based on De Morgan's Laws or other considerations.

~~~~~ Short-circuit evaluation ~~~~~

An important thing to remember about the logical operators `&&` and `||` is *short-circuit evaluation*. If the value of the first operand is sufficient to determine the result, then the second operand is not evaluated.

5

Consider the following code segment.

```
int x = 0, y = 3;
String op = "/";

if (op.equals("/") && (x != 0) && (y/x > 2))
{
    System.out.println("OK");
}
else
{
    System.out.println("Failed");
}
```

Which of the following statements about this code is true?

- (A) There will be a compile error because `String` and `int` variables are intermixed in the same condition.
- (B) There will be a run-time divide-by-zero error.
- (C) The code will compile and execute without error; the output will be `OK`.
- (D) The code will compile and execute without error; the output will be `Failed`.

 Choice A is just a filler answer. Since `x` is equal to 0, the condition cannot be true, so Choice C should be rejected, too. The question remains whether it crashes or executes. In Java, once `x != 0` fails, the rest of the condition, `y/x > 2`, won't be evaluated, and `y/x` won't be computed. The answer is D. 

The relational expressions in the above question are parenthesized. This is not necessary because relational operators always take precedence over logical operators. If you are used to lots of parentheses, use them, but you can skip them as well. For example, the Boolean expression from Question 5 can be written with fewer parentheses:

```
if (op.equals("/") && x != 0 && y/x > 2) ...
```

`&&` also takes precedence over `||`, but it's clearer to use parentheses when `&&` and `||` appear in the same expression. For example:

```
if ((0 < a && a < top) || (0 < b && b < top)) ...
```

Bitwise logical operators

The bitwise logical operators, `&`, `|`, `^`, and `~`, are not in the AP Java subset and are not tested on the AP exam. You don't need to worry about them.

Programmers use these operators to perform logical operations on individual bits, usually in `int` values. Unfortunately, Java also allows you to apply these operators to `boolean` values, and, when used that way, these operators do not comply with short-circuit evaluation. This may lead to a nasty bug if you inadvertently write `&` instead of `&&`, or `|` instead of `||`. For example,

```
if (x != 0 & y/x > 2)
```

Error: `&` instead
of `&&`



results in a division by 0 exception when `x = 0`.

2.2 Conditional Statements and Loops

You can use simplified indentation for `if-else-if` statements.

For example:

```
if (score >= 70)
    grade = 5;
else if (score >= 60)
    grade = 4;
...
else
    grade = 1;
```

But don't forget braces and proper indentation for nested `ifs`. For example:

```
if (exam.equals("Calculus AB"))
{
    if (score >= 60)
        grade = 5;
    else if ...
        ...
}
else if (exam.equals("Calculus BC"))
{
    if (score >= 70)
        grade = 5;
    else if ...
        ...
}
```


6

Consider the following code segment, in which *m* is a variable of the type `int`.

```
if (m > 0)
{
    if ((1000 / m) % 2 == 0)
        System.out.println("even");
    else
        System.out.println("odd");
}
else
    System.out.println("not positive");
```

Which of the following code segments is equivalent to the one above — that is, produces the same output as the one above regardless of the value of *m*?

(A)

```
if (m < 0)
    System.out.println("not positive");
else if (m != 0 && (1000 / m) % 2 == 0)
    System.out.println("even");
else
    System.out.println("odd");
```

(B)

```
if (m > 0 && (1000 / m) % 2 == 0)
    System.out.println("even");
else if (m <= 0)
    System.out.println("not positive");
else
    System.out.println("odd");
```

(C)

```
if ((1000 / m) % 2 == 0)
{
    if (m <= 0)
        System.out.println("not positive");
    else
        System.out.println("even");
}
else
{
    if (m <= 0)
        System.out.println("not positive");
    else
        System.out.println("odd");
}
```

(D) None of the above

 If *m* = 0, the code in Choice A prints nothing and the code in Choice C will “throw” an `ArithmeticException` due to division by zero. The code in Choice B is correctly restructured original code. The answer is B. 

~~~~~ for *and* while loops ~~~~~

The for loop,

```
for (initialize; condition; change)
{
    ...    // Do something
}
```

is equivalent to the while loop:

```
initialize;
while (condition)
{
    ...    // Do something
    change;
}
```

*change* can mean any change in the values of the variables that control the loop, such as incrementing or decrementing an index or a counter.

for loops are shorter and more idiomatic than while loops in many situations. For example:

```
int i = 0;
while (i < a.length)
{
    sum += a[i];
    i++;
}
```

OK

```
for (int i = 0; i < a.length; i++)
    sum += a[i];
```

Better, more idiomatic

**In a for or while loop, the condition is evaluated at the beginning of the loop and the program does not go inside the loop if the condition is false. Thus, the body of the loop will be skipped entirely if the condition is false at the very beginning.**

**7**

Consider the following methods.

```
public int fun1(int n)
{
    int product = 1;
    for (int k = 2; k <= n; k++)
    {
        product *= k;
    }
    return product;
}
```

```
public int fun2(int n)
{
    int product = 1;
    int k = 2;
    while (k <= n)
    {
        product *= k;
        k++;
    }
    return product;
}
```

For which integer values of  $n$  do  $\text{fun1}(n)$  and  $\text{fun2}(n)$  return the same result?

- (A) Only  $n \geq 1$
- (B) Only  $n < 1$
- (C) Only  $n \leq 2$
- (D) Any integer  $n$

 The best approach here is purely formal: since the initialization, condition, and increment in the `for` loop in `fun1` are the same as the ones used with the `while` loop in `fun2`, the two methods are equivalent. The answer is D. 

**8**

Consider the following code segment.

```
while (x > y)
{
    x--;
    y++;
}
System.out.println(x - y);
```

Assume that  $x$  and  $y$  are `int` variables and their values satisfy the conditions  $0 \leq x \leq 2$  and  $0 \leq y \leq 2$ . Which of the following describes the set of all possible outputs?

- (A) 0
- (B) -1, -2
- (C) 0, -1, -2
- (D) 0, -1, 1, -2, 2

✚ If  $x \leq y$  to begin with, then the `while` loop is never entered and the possible outputs are 0, -1, and -2 (for the pairs (0, 0), (0, 1), (0, 2), (1, 1), (1, 2), (2, 2)). If  $x > y$ , then the loop is entered and after the loop we must have  $x \leq y$ , so  $x - y$  cannot be positive. The answer is C. ✚

## ~~~~~ OBOBS ~~~~~

**When coding loops, beware of the so-called “off-by-one bugs” (“OBOBS”). These are mistakes of running through the loop one time too many or one time too few.**

**9**

Suppose the `isPrime` method is defined:

```
// Returns true if p is a prime, false otherwise.
// Precondition: p >= 2
public static boolean isPrime(int p) { < implementation not shown > }
```

Given

```
int n = 101;
int sum = 0;
```

which of the following code segments correctly computes the sum of all prime numbers from 2 to 101, inclusive?

- (A) 

```
while (n >= 2)
{
    n--;
    if (isPrime(n)) sum += n;
}
```
- (B) 

```
while (n != 2)
{
    if (isPrime(n)) sum += n;
    n--;
}
```
- (C) 

```
while (n >= 2)
{
    if (isPrime(n)) sum += n;
    n--;
}
```
- (D) 

```
while (n >= 2 && isPrime(n))
{
    sum += n;
    n--;
}
```

🔗 It is bad style to start the body of a loop with a decrement, so Choice A is most likely wrong. Indeed, it misses 101 (which happens to be a prime) because *n* is decremented too early. Choice B misses 2 — an OBOB on the other end. Choice D might look plausible for a moment, but it actually quits as soon as it encounters the first non-prime number. The answer is C. 📌

### ~~~~~ *The for-each loop* ~~~~~

The for-each loop (also known as “enhanced for loop”) has the form

```
for (SomeType x : a) // read: "for each x in a"
{
    ... // do something
}
```

where *a* is an array or an `ArrayList` (or another type of list or “collection”) that holds values of *SomeType*.

For example:

```
int[] scores = {87, 95, 76};
for (int sc : scores)
    System.out.print(sc + " ");
```

works the same way as

```
int[] scores = {87, 95, 76};
for (int i = 0; i < scores.length; i++)
{
    int sc = scores[i];
    System.out.print(sc + " ");
}
```

Both will display

87 95 76

Another example:

```
ArrayList<String> plants = new ArrayList<String>();
plants.add("Bougainvillea");
plants.add("Hibiscus");
plants.add("Poinciana");

for (String name : plants)
    System.out.print(name + " ");
```

will display

```
Bougainvillea Hibiscus Poinciana
```

Notice that the for-each loop traverses the array or list only in the forward direction and does not give you access to the indices of the values stored in the array or list. For example, a for-each loop won't be very useful if you need to find the position of the first occurrence of a target value in a list.

**If you need access to indices, use a for or while loop.**

**A for-each loop does not allow you to set an element of an array to a new value, because it works with a copy of the element's value, and so the original values of an array will remain unchanged.**

For example, given an int array arr,

```
for (int x : arr)
    x = 1;
```

will leave arr unchanged.

**If you need to replace or delete elements of an array or ArrayList, use an indexed (regular) for or while loop.**

If an array or ArrayList holds objects, then you can call an object's method inside a for-each loop, so you can potentially change an object's state if the method changes it.

You cannot add or remove elements from an ArrayList while traversing it with a for-each loop.

**If you attempt to add an element to an ArrayList or remove an element from an ArrayList while traversing it with a for-each loop, your code will throw a ConcurrentModificationException.**

10

What is the output from the following code segment?

```
ArrayList<String> months = new ArrayList<String>();
months.add("Jan");
months.add("Feb");
months.add("Mar");
for (String mo : months)
    mo = mo.substring(1);
System.out.println(months);
```

- (A) [J, F, M]
- (B) [an, eb, ar]
- (C) [Jan, Feb, Mar]
- (D) ConcurrentModificationException

 As explained above, `mo` in the for-each loop here is a copy of a reference to an element of the `ArrayList` `months`, so setting `mo` to a different value, whatever that might be, won't change the list. `ConcurrentModificationException` happens only if you add or remove elements in a for-each loop. The answer is C. 

### ~~~~~ break *and* return *in* loops ~~~~~

In Java it is okay to use `break` and `return` inside loops. `return` immediately quits the method from any place inside or outside a loop. This may be a convenient shortcut, especially when you have to quit a method from within a nested loop. For example:

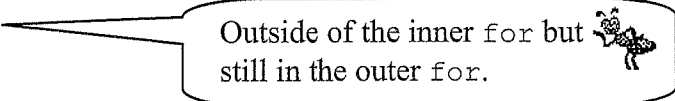
```
// Returns true if all values in list are different, false otherwise
public boolean allDifferent(int[] list)
{
    for (int i = 0; i < list.length; i++)
    {
        for (int j = i + 1; j < list.length; j++)
        {
            if (list[i] == list[j])
                return false;
        }
    }
    return true;
}
```

 You can also use `break`, but it may be dangerous and is not in the AP Java subset. Remember that in a nested loop, `break` takes you out of the inner loop but not out of the outer loop.

Avoid redundant, verbose, and incorrect code such as this:

```
// Returns true if all values in list are different, false otherwise
public boolean allDifferent(int[] list)
{
    boolean foundDuplicates;

    for (int i = 0; i < list.length; i++)
    {
        for (int j = i + 1; j < list.length; j++)
        {
            if (list[i] == list[j])
            {
                foundDuplicates = true;
                break;
            }
            else
            {
                foundDuplicates = false;
            }
        }
    }
    if (foundDuplicates == true)
        return false;
    else
        return true;
}
```



If you insist on using boolean flags, you need to be extra careful:

```
// Returns true if all values in list are different, false otherwise
public boolean allDifferent(int[] list)
{
    boolean foundDuplicates = false;

    for (int i = 0; i < list.length && !foundDuplicates; i++)
    {
        for (int j = i + 1; j < list.length; j++)
        {
            if (list[i] == list[j])
            {
                foundDuplicates = true;
                break;
            }
        }
    }
    return !foundDuplicates;
}
```

The continue statement is not in the AP Java subset and should be avoided.



## 2.3 Strings

In Java, a string is an object of the type `String`, and, as with other types of objects, a `String` variable holds a reference to (the memory address of) the string.

**Strings are immutable: no method of `String` can change the string.**

An assignment statement

```
str1 = str2;
```

copies the reference from `str2` into `str1`, so they both refer to the same memory location.

A *literal string* is a string of characters within double quotes. A literal string may include the “escape characters” `\n` (newline), `\` (a double quote), `\\` (one backslash), and other escapes. For example,

```
System.out.print("Hello\n");
```

has the same effect as

```
System.out.println("Hello");
```

The `String` class supports the `+` and `+=` operators for concatenating strings.

**`String` is the only class in Java that supports special syntax for using operators on its objects (only `+` and `+=`).**

The operator

```
s1 += s2;
```

appends `s2` to `s1`. In reality, it creates a new string by concatenating `s1` and `s2` and then sets `s1` to refer to the new string. It is equivalent to

```
s1 = s1 + s2;
```

11

What is the output from the following code segment?

```
String str1 = "Happy ";
String str2 = str1;
str2 += "New Year! ";
str2.substring(6);
System.out.println(str1 + str2);
```

- (A) Happy New Year!
- (B) Happy Happy New Year!
- (C) Happy New Year! New Year!
- (D) Happy New Year! Happy

➦ After `str2 = str1`, `str1` and `str2` point to the same memory location that contains "Happy ". But after `str2 += "New Year! "`, these variables point to different things: `str1` remains "Happy " (strings are immutable), while `str2` becomes "Happy New Year! ". `str2.substring(6)` does not change `str2` — it calls its `substring` method but does not use the returned value (a common beginners' mistake; again, strings are immutable). The answer is B. ➦

### ~~~~~ String *methods* ~~~~~

The String methods included in the AP Java subset are:

```
int length()
String substring(int from, int to)
String substring(int from)
int indexOf(String s)
boolean equals(Object other) [ML: State the methods in the JQR order]
int compareTo(String other)
String[] split(String del)
```

**Always use the `equals` method to compare a string to another string. The `==` and `!=` operators, applied to two strings, compare their addresses, not their values.**

`str.substring(from, to)` returns `str`'s substring starting at the `from` position and up to but not including the `to` position (so the length of the returned substring is `to minus from`). For example, `"Happy".substring(1, 4)` would return `"app"`.

### Positions are counted from 0.

`str.substring(from)` returns a substring of `str` from the `from` position to the end. For example, `"Spring".substring(2)` would return `"ring"`. If `from` is equal to `str.length()`, then `str.substring(from)` returns an empty string. If `from` is greater than `str.length()`, then `str.substring(from)` throws a `StringIndexOutOfBoundsException`.

`str.indexOf(s)` returns the starting position of the first occurrence of the substring `s` in `str`, or `-1` if not found.

`str1.equals(str2)` returns `true` if and only if `str1` and `str2` have the same values (that is, consist of the same characters in the same order).

`str1.compareTo(str2)` returns a positive number if `str1` is greater than `str2` (lexicographically), zero if they are equal, and a negative number if `str1` is less than `str2`. Uppercase letters are “smaller” than (come before) lowercase letters, so `"ABC".compareTo("abc")` returns a negative value.

`str.split(delim)` takes a `String` as a parameter and returns an array of `Strings` that are formed when `str` is split around matches to `delim`. For example,

```
String[] fruit = "Apple, Banana, Pear".split(", ");
```

creates an array `fruit` of three elements with values `"Apple"`, `"Banana"`, and `"Pear"`.

 An argument to `String`'s `split` methods actually can be a *regular expression* — a string with escape characters that defines a pattern that matches a certain class of strings. Regular expressions are used for parsing text; they have their own syntax and rules and are supported by most modern programming languages. An ordinary string can be viewed as a regular expression that matches only identical strings. In the AP Java subset, only ordinary strings are used as arguments to the `split` method.

 You don't need to understand or use regular expressions in AP exams.

**12**

Consider the following method.

```
public String process(String msg, String delim)
{
    int pos = msg.indexOf(delim);
    while (pos >= 0)
    {
        msg = msg.substring(0, pos) + " "
              + msg.substring(pos + delim.length());
        pos = msg.indexOf(delim);
    }
    return msg;
}
```

What is the output from the following code segment?

```
String rhyme = "Twinkle\ntwinkle\nlittle star";
String rhyme2 = process(rhyme, "\n");
System.out.println(rhyme + "\n" + rhyme2);
```

- (A) little star  
Twinkle twinkle little star
- (B) little star  
Twinkle  
twinkle  
little star
- (C) Twinkle  
twinkle  
little star  
Twinkle winkle ittle tar
- (D) Twinkle  
twinkle  
little star  
Twinkle twinkle little star

 `process` receives and works with a copy of a reference to the original string (see Section 3.3). The method can reassign the copy, as it does here, but the original reference still refers to the same string. This consideration, combined with the immutability of strings, assures us that `rhyme` remains unchanged after the call `process(rhyme)`.

The rhyme string includes two newline characters, and, when printed, it produces

```
Twinkle
twinkle
little star
```

So, the only possible answers are Choices C or D. Notice that we can come to this conclusion before we even look at the `process` method! This method repeatedly finds the first occurrence of `delim` in `msg`, cuts it out, and replaces it with a space. No other characters are replaced or lost. The resulting message prints on one line. The answer is D. 

## 2.4 Integer and Double Classes

In Java, variables of primitive data types (`int`, `double`, and so on) are not objects. In some situations it is convenient to represent numbers as objects. For example, you might want to store numeric values in an `ArrayList` (see Section 2.6), but the elements of an `ArrayList` must be objects. The `java.lang` package provides several “wrapper” classes that represent primitive data types as objects. Two of these classes, `Integer` and `Double`, are in the AP Java subset.

 The `Integer` class has a constructor that takes an `int` value and creates an `Integer` object representing that value. The `intValue` method of an `Integer` object returns the value represented by that object as an `int`. For example:

```
Integer obj = new Integer(123);
...
int num = obj.intValue(); // num gets the value of 123
```

Likewise, `Double`’s constructor creates a `Double` object that represents a given double value. The method `doubleValue` returns the double represented by a `Double` object:



```
Double obj = new Double(123.45);
...
double x = obj.doubleValue(); // x gets the value of 123.45
```

The `Integer` class has a static method `parseInt` that takes a string representation of an integer and returns the corresponding `int` value. For example,

```
int x = Integer.parseInt("123"); // x gets the value 123
```

Similarly, `Double.parseDouble("1.23")` gives 1.23.

The `Integer` class also provides two symbolic constants that describe the range for `int` values:

```
public static final int MIN_VALUE = -2147483648;
public static final int MAX_VALUE = 2147483647;
```

**Use the `equals` method of the `Integer` or of the `Double` class if you want to compare two `Integer` or two `Double` variables, respectively.**

For example:

```
Integer a = new Integer(...);
Integer b = new Integer(...);
...
if (a.equals(b))
    ...
```

**If you apply a relational operator (`==` or `!=`) to two `Integer` or two `Double` variables, you will compare their addresses, not their values.**

This is rarely, if ever, what you want to do.

Both `Integer` and `Double` classes have a `compareTo` method. As usual, `obj1.compareTo(obj2)` returns a positive integer if `obj1` is greater than `obj2` (that is, `obj1`'s numeric value is greater than `obj2`'s numeric value), a negative integer if `obj1` is less than `obj2`, and zero if their numeric values are equal.

## *Autoboxing/unboxing*

The Java compiler in certain situations automatically converts values of primitive data types (`int`, `double`, and so on) into the corresponding wrapper types (`Integer`, `Double`, and so on). This feature is called *autoboxing*. For example, in

```
ArrayList<Integer> numbers = new ArrayList<Integer>();
numbers.add(5);
```

the second line is compiled as

```
numbers.add(new Integer(5));
```

Likewise, where appropriate, the compiler performs *unboxing*. For example,

```
int num = numbers.get(0);
```

in effect compiles as

```
int num = numbers.get(0).intValue();
```

**Autoboxing and unboxing are included in the AP Java subset but are unlikely to be tested in isolation.**

## 2.5 Arrays

There are two ways to declare and create a one-dimensional array:

```
SomeType[] a = new SomeType[size];  
SomeType[] b = {value0, value1, ..., valuen-1};
```

For example:

```
double[] samples = new double[100];  
int[] numbers = {1, 2, 3};  
String[] cities = {"Atlanta", "Boston", "Cincinnati", "Dallas"};
```

In this example, the first statement declares an array of doubles of size 100. Its elements get default values (zeros). The second statement creates an array of ints of size 3 with its elements initialized to the values 1, 2, and 3. The third statement declares and initializes an array of four given strings.

We can refer to a's elements as `a[i]`, where `a` is the name of the array and `i` is an index, which can be an integer constant, variable, or expression.

**Indices start from 0.**

**`a.length` refers to the size of the array.**

(In an array, `length` is not a method, but rather works like a public instance variable, hence no parentheses.)

**`a[a.length-1]` refers to the last element.**

Once an array is created, its size cannot be changed. The only way to expand an array is to create a bigger array and copy the contents of the original array into the new one.

The old array is discarded (or, more precisely, recycled through a process called *garbage collection*).

For example:

```
int[] a = new int[100];
...
int[] temp = new int[a.length * 2];

for (int i = 0; i < a.length; i++)
    temp[i] = a[i];

a = temp;    // reassign a to the new array; the old array is discarded
```

**If *a* and *b* are arrays, *a* = *b* does not copy elements from *b* into *a*; it just reassigns the reference *a* to *b*, so that both *a* and *b* refer to the same array.**

The following method reverses the order of elements in an array of strings:

```
public void reverse(String[] words)
{
    int i = 0, j = words.length - 1;

    while (i < j)
    {
        String temp = words[i]; words[i] = words[j]; words[j] = temp;
        i++;
        j--;
    }
}
```

The Java Virtual Machine (the run-time interpreter) checks that an array index is within the valid range, from 0 to `array.length-1`. If an index value is invalid, the interpreter throws an `ArrayIndexOutOfBoundsException` — reports a run-time error, the line number for the offending program statement, and a trace of the method calls that led to it.

**An *exception* is a run-time error, not a compile-time error.**

The compiler cannot catch errors that will be caused by certain circumstances that occur during program execution, such as a variable used as an array index whose value has gone out of the allowed range.



**13**

Suppose the method `int sign(int x)` returns 1 if `x` is positive, -1 if `x` is negative, and 0 if `x` is 0. Given

```
int[] nums = {-2, -1, 0, 1, 2};
```

what are the values of the elements of `nums` after the following code is executed?

```
for (int k = 0; k < nums.length; k++)
{
    nums[k] -= sign(nums[k]);
    nums[k] += sign(nums[k]);
}
```

- (A) -2, -1, 0, 1, 2
- (B) -1, 0, 0, 0, 1
- (C) 0, 0, 0, 0, 0
- (D) -2, 0, 0, 0, 2

 Remember that the first statement within the loop changes `nums[k]`, which may change the sign of `nums[k]`, too. Jot down a little table:

| <i>Before</i> |                           | <i>After -=</i> |                           | <i>After +=</i> |
|---------------|---------------------------|-----------------|---------------------------|-----------------|
| <i>a[k]</i>   | sign<br>of<br><i>a[k]</i> | <i>a[k]</i>     | sign<br>of<br><i>a[k]</i> | <i>a[k]</i>     |
| -2            | -1                        | -1              | -1                        | -2              |
| -1            | -1                        | 0               | 0                         | 0               |
| 0             | 0                         | 0               | 0                         | 0               |
| 1             | 1                         | 0               | 0                         | 0               |
| 2             | 1                         | 1               | 1                         | 2               |

The answer is D. 

**14**

Consider the following method.

```
/**
 * Returns true if there are no two elements among
 * counts[0], ... counts[n-1] whose values are the same
 * or are consecutive integers; otherwise, returns false.
 * Preconditions: counts contains n values, n > 1
 */
public boolean isSparse(int[] counts, int n)
{
    < missing code >
}
```

Which of the following code segments cannot be used to replace < missing code > so that the method `isSparse` works as specified?

(A)

```
for (int j = 0; j < n; j++)
{
    for (int k = j + 1; k < n; k++)
    {
        int diff = counts[j] - counts[k];
        if (diff >= -1 && diff <= 1)
            return false;
    }
}
return true;
```

(B)

```
for (int j = 1; j < n; j++)
{
    for (int k = 0; k < j; k++)
    {
        int diff = counts[j] - counts[k];
        if (diff >= -1 && diff <= 1)
            return false;
    }
}
return true;
```

(C)

```
for (int j = 0; j < n; j++)
{
    for (int k = 1; k < n; k++)
    {
        int diff = counts[j] - counts[k];
        if (Math.abs(diff) <= 1)
            return false;
    }
}
return true;
```

(D) All of the above will work

☞ Notice that in this question not all `counts.length` elements of `counts` are used, just the first `n`. Their subscripts range from 0 to `n-1`. The precondition states that `n > 1`, so there is no need to worry about an empty array or an array of just one element. Looking inside the inner loop in each segment you can quickly see that they work the same way. So, the difference is how the loops are set up — more precisely, the limits within which the indices vary. In Choice A the outer loop starts with the first item in the list; the inner loop compares it with each of the subsequent items. In Choice B the outer loop starts with the second item in the list; the inner loop compares it with each of the preceding items. Both of these are correct and quite standard in similar algorithms. This eliminates Choices A and B. Choice C at first seems harmless, too, but it has a catch: the inner loop doesn't set a limit for `k` that depends on `j`, so when `j` is greater than 0, `k` may eventually take the same value as `j` (such as `j = 1, k = 1`). The method will erroneously detect the same value in `counts` when it is actually comparing an item to itself. The answer is C. ☞

### ~~~~~ *Array return type* ~~~~~

Occasionally you may need to return an array from a method. Suppose you want to restructure the `reverse` method above so that it returns a new array containing the values from a given array in reverse order. The original array remains unchanged. The method can be coded as follows:

```
public String[] reverse(String[] words)
{
    String[] result = new String[words.length];

    for (int i = 0; i < words.length; i++)
        result[i] = words[words.length - 1 - i];

    return result;
}
```

### ~~~~~ *Two-dimensional arrays* ~~~~~

The AP Java subset includes only rectangular two-dimensional arrays. These are similar to one-dimensional arrays but use two indices, one for the row and one for the column. For example:

```
double[][] matrix = new double[3][5]; // 3 rows by 5 cols
int r, c;
...
matrix[r][c] = 1.23;
```

**In fact, a two-dimensional array is an array of arrays, its rows. If `m` is a two-dimensional array, `m[0]` is its first row, `m[1]` is the second row, and so forth.**

Therefore,

**if `m` is a two-dimensional array, `m.length` represents the number of rows.**

`m[0].length` represents the length of the first row.

**Only rectangular 2D arrays are considered in the AP Java subset; therefore, the lengths of all the rows are the same, so `m[0].length` represents the number of columns in every row.**

The following method calculates the sums of the values in each column of a 2D array, places these sums into a new 1D array of sums, and returns that array:

```
// Returns a 1D array containing sums of all the values
// in each column of table
public double[] totalsByColumn(double[][] table)
{
    int nRows = table.length;
    int nCols = table[0].length;

    double[] totals = new double[nCols];

    for (int c = 0; c < nCols; c++)
    {
        totals[c] = 0.0;    // optional: default

        for (int r = 0; r < nRows; r++)
            totals[c] += table[r][c];
    }
    return totals;
}
```

**15**

Consider the following code segment.

```
String[][] m = new String[6][3];  
for (int k = 0; k < m.length; k++)  
{  
    m[k][m[0].length - 1] = "*";  
}
```

Which of the following best describes the result when this code segment is executed?

- (A) All elements in the first row of `m` are set to `"*"`
- (B) All elements in the last row of `m` are set to `"*"`
- (C) All elements in the last column of `m` are set to `"*"`
- (D) `ArrayIndexOutOfBoundsException` is reported

 The first index is the row, and the `for` loop is set up for all rows. The answer is C. 

 Since a two-dimensional array is an array of one-dimensional arrays, a single for-each loop won't work for traversing a 2D array.

To make a for-each loop work on a 2D array, you need nested for-each loops. For example:

```
int[][] a = {{1, 2, 3}, {4, 5, 6}};  
for (int[] row : a)  
{  
    for (int x : row)  
        System.out.print(x + " ");  
    System.out.println();  
}
```

## 2.6 The ArrayList Class

`java.util.ArrayList<E>` is a Java library class that defines a list of objects of some type *E*.

Another Java library class that represents a list, `java.util.LinkedList<E>`, is not included in the AP Java subset and not tested on AP CSA exams.

For example, assuming the class `Student` is defined,

```
ArrayList<Student> list = new ArrayList<Student>();
```

defines an `ArrayList` of `Student` objects.

**An `ArrayList` cannot hold values of a primitive data type. For instance, `ArrayList<int>` or `ArrayList<double>` will result in a syntax error.**

However, an `int` or `double` value can be added to an `ArrayList<Integer>` or `ArrayList<Double>`, respectively, due to autoboxing. For example:

```
ArrayList<Double> prices = new ArrayList<Double>();  
prices.add(29.95);
```

`ArrayList` provides methods for getting and setting the value of a particular element, adding a value at the end of the list, removing a value, and inserting a value at a given position. As in standard arrays, indices start from 0.

An `ArrayList` is automatically resized when it runs out of space for added elements. `ArrayList`'s “no-args” constructor (that is, the constructor that takes no arguments) allocates an array of some default initial capacity and sets the size to 0 (no values stored in the `ArrayList` yet). As values are added, their number may exceed the current capacity. Then a new array is allocated with twice the capacity, the old values are copied into the new array, and the old array is discarded. All this happens behind the scenes — you don't have to worry about any of it.

The AP Java subset includes the following methods of `ArrayList<E>`:

|                                         |                                                                                                                                                                                                                                                     |
|-----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>int size()</code>                 | Returns the number of elements in the list                                                                                                                                                                                                          |
| <code>boolean add(E obj)</code>         | Adds <code>obj</code> at the end of the list; returns <code>true</code>                                                                                                                                                                             |
| <code>void add(int index, E obj)</code> | Inserts <code>obj</code> at position <code>index</code> ( $0 \leq \text{index} \leq \text{size}$ ), shifting the current element stored at <code>index</code> and all the subsequent elements toward the end of the list; increases the size by one |
| <code>E get(int index)</code>           | Returns the element at position <code>index</code> in the list                                                                                                                                                                                      |
| <code>E set(int index, E obj)</code>    | Replaces the element at position <code>index</code> with <code>obj</code> ; returns the element formerly at position <code>index</code>                                                                                                             |
| <code>E remove(int index)</code>        | Removes the element from position <code>index</code> , shifting the subsequent elements toward the beginning of the list; reduces the size by one; returns the element formerly at position <code>index</code>                                      |

The `add` and `remove` methods adjust the size of the `ArrayList` appropriately. The methods that take an `index` parameter check that the index is in the valid range, from 0 to `size() - 1` (or 0 to `size()` for `add(index, obj)`), and *throw* an `IndexOutOfBoundsException` if the index is not in that range.

It might be helpful for you to know and use on the AP exam three additional methods of `ArrayList`, which are not in the AP Java subset:

|                                           |                                                                                                                                                                |
|-------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>boolean contains(Object obj)</code> | Returns <code>true</code> if one of the elements in the list is equal to <code>obj</code>                                                                      |
| <code>int indexOf(Object obj)</code>      | Returns the index of the first occurrence of <code>obj</code> in the list, or <code>-1</code> if the list does not contain <code>obj</code>                    |
| <code>boolean remove(Object obj)</code>   | If the list contains <code>obj</code> , removes the first occurrence of <code>obj</code> and returns <code>true</code> ; otherwise, returns <code>false</code> |

Since the compiler “knows” what type of objects is stored in an `ArrayList`, it automatically casts values retrieved from the list into their type. For example:

```
ArrayList<Fish> list = new ArrayList<Fish>();  
...  
Fish f = list.get(i);
```

**16**

What is the output from the following code segment?

```
ArrayList<String> list = new ArrayList<String>();  
list.add("A");  
list.add("B");  
list.add("C");  
list.add("D");  
list.add("E");  
  
for (int k = 1; k <= 3; k++)  
{  
    list.remove(1);  
}  
  
for (int k = 1; k <= 3; k++)  
{  
    list.add(1, "*");  
}  
  
for (String word : list)  
{  
    System.out.print(word + " ");  
}
```

- (A) \* \* \* B C D E
- (B) A \* \* \* E
- (C) A E \* \* \*
- (D) IndexOutOfBoundsException

 This question is not as tricky as it might seem. First we create an empty list and add five values to it: "A", "B", "C", "D", "E". Then we remove the value at index 1 three times. This is the second element, and each time we remove it, the subsequent values are shifted toward the beginning by one position. "A" and "E" remain. Then we insert three asterisks. Notice that we always insert at index 1. After the first insertion we get "A", "\*", "E". After the second we get "A", "\*", "\*", "E". The third insertion produces "A", "\*", "\*", "\*", "E". The third loop (a for-each loop) traverses the list and prints out the values. The answer is B. 



## 2.7 Exceptions

An exception is a run-time event that signals an abnormal condition in the program. Some run-time errors, such as invalid user input or an attempt to read past the end of a file, are considered fixable. The `try-catch-finally` syntax allows the programmer to catch and process the exception and have the program recover. This type of exception is called a *checked exception*.

**Checked exceptions and the `try-catch` statements are not in the Java subset and won't be tested.**

Other errors, such as an array index out of bounds or an attempt to call a method of a non-existing object (null reference) are considered fatal: the program displays an error message with information about where the error occurred, then quits. This type of exception is called an *unchecked exception*.

In Java, an exception is an object of one of the Java exception classes. The Java library implements many types of exceptions, and, if necessary, you can derive your own exception class from one of the library classes. For an AP CSA exam, you are expected to understand when `ArithmeticException`, `IndexOutOfBoundsException`, `ArrayIndexOutOfBoundsException`, `StringIndexOutOfBoundsException`, `NullPointerException`, `ConcurrentModificationException` and `IOException` happen.

We say that a program “throws” an exception.

An `ArithmeticException` is thrown when an arithmetic error, such as integer division by zero, occurs. You would expect `Math.sqrt(x)` to throw an `ArithmeticException` for a negative `x`, but it doesn't — it returns `NaN`, “not a number,” instead. This situation won't be tested. Java exception handling is inconsistent at times.

`ArrayIndexOutOfBoundsException` is self-explanatory: it is thrown at run time when an array index is negative or is greater than `array.length-1`. `String` methods may throw a similar `StringIndexOutOfBoundsException`, and `ArrayList` methods throw a similar `IndexOutOfBoundsException`.

A `NullPointerException` is thrown when you forget to initialize an object-type variable or an element of an array and then try to call its method. For example:

```
public class MyClass
{
    private String name; // name is set to null
    ...
    // the following statement is inside a method:
    int n = name.length(); // if name has not been initialized
                          // by MyClass's constructor, this statement
                          // will throw a NullPointerException
    ...
}
```

Another example:

```
public class DeckOfCards
{
    private Card[] cards;

    public DeckOfCards()
    {
        cards = new Card[52]; // all elements set to null
        // Forgot to initialize each element
    }

    public Card getCard(int k) { return cards[k]; }
}
```

In a client class,

```
DeckOfCards deck = new DeckOfCards();
int r = deck.getCard(0).getRank();
```

will throw a `NullPointerException`, because `deck.getCard(0)` returns `null`.

A `ConcurrentModificationException` is thrown if you attempt to add or remove elements of an `ArrayList` while traversing it in a `for-each` loop.

**17**

Consider the following method.

```
public void removeConsecutiveDuplicates(ArrayList<String> names)
{
    int n = names.size();
    String prev = null;
    for (int i = 0; i < n; i++)
    {
        String name = names.get(i);
        if (name.equals(prev))
            names.remove(i);
        prev = name;
    }
}
```

What is the result when the method is called on an ArrayList that holds ["Alice", "Bob", "", "", ""]?

- (A) IndexOutOfBoundsException
- (B) ConcurrentModificationException
- (C) NullPointerException
- (D) The method won't throw an exception but will miss one of the empty strings.

 A ConcurrentModificationException may happen only if you add or remove elements in a for-each loop — not in a regular indexed loop. A NullPointerException won't happen here because equals is never called on a null object (an empty string is not null). equals(null) on the first iteration is okay. Removal of an element will shorten the list, while the index i goes all the way to the original list size. The answer is A. 

## 2.8 print and println

In the AP subset, output is limited to System.out.print and System.out.println calls.

You don't have to worry about formatting numbers. Java converts an int or a double value passed to System.out.print or System.out.println into a string using default formatting.

It is unusual for the System.out.print and System.out.println methods to be required in solutions to free-response questions. (This happened only once prior to the time of this writing, in the 2004 exam.)

**Never use `print` or `println` in your solutions unless it is explicitly requested with the word “print.”**

↓ The `System.out.printf` method with a variable number of parameters can be used for more precisely formatted output of one or several numbers and strings.

↑ **`printf` is not in the AP Java subset and should not be used in your exam solutions.**

You can pass any object to `System.out.print` or `System.out.println`. These methods handle an object by calling its `toString` method. Both `Integer` and `Double` classes have reasonable `toString` methods defined.

↓ In general, if you are designing a class, it is a good idea to supply a reasonable `toString` method for it. For example:

```
public class Fraction
{
    ...
    public String toString()
    {
        return num + "/" + denom;
    }
}
```

↑ Otherwise, your class inherits a generic `toString` method from `Object`, which returns the object's class name followed by the object's address.

**Implementing a `toString` method is not required in any of the AP exam's free-response questions, including Question 2.**

So, don't do that — it won't earn you any extra points, and it may result in a deduction if you don't do it correctly.

**The `System.out.print` and `System.out.println` methods take only one parameter.**

If you need to print several things, use the `+` operator for concatenating strings. You can also concatenate a string and an `int` or `double`; the latter will be converted into a string. For example:

```
System.out.println(3 + " hours " + 15 + " minutes.");
```

The displayed result will be

3 hours 15 minutes.

You can also concatenate a string and an object; the object's `toString` method will be called to convert it into a string. For example:

```
int n = 3, d = 4;
Fraction f = new Fraction(n, d);
System.out.println(f + " = " + (double)n / (double)d);
```

The displayed result will be

3/4 = 0.75

Just be careful not to apply a `+` operator to two numbers or two objects other than strings; in the former case, the numbers will be added rather than concatenated, whereas the latter case will result in a syntax error.

## 2.9 Reading Text Files

This topic has been added to the CED effective Fall 2025. The material is limited to reading text files by using the `File` and `IOException` classes from the `java.io` package and the `Scanner` class from the `java.util` package.

The only tested `File` constructor takes one `String` parameter, the name (more precisely, the *pathname*) of the file. Invoking the constructor creates a file descriptor object, even if the file does not exist or is not found.

The relevant `Scanner` constructor takes one `File` object parameter and attempts to open that file for reading. This constructor throws an `IOException` (more precisely, the `FileNotFoundException`) if the file is not found.\*

---

\* The CED, published in March of 2025, erroneously stated that the `File` constructor throws `IOException`. It is the `Scanner(File f)` constructor that does.

↓ This is a *checked exception* that normally would be caught in the program using the try-catch block. For example:

```
Scanner myText = null;
try
{
    myText = new Scanner(new File("data.txt"));
}
catch FileNotFoundException
{
    System.out.println("data.txt file not found")
}
...
```

However, the Exam Development Committee decided to exclude the handling of checked exceptions, and try-catch syntax, from the AP Java subset. The

↑ `FileNotFoundException` is excluded, too.

Instead, the CED suggests adding `throws IOException` to the header of a method that opens the file. The problem with this approach is that `throws IOException` then has to be added to the calling method and all the way up to main. No knowledgeable coder would do that. It seems unlikely, anyway, that a requirement to read a text file will appear in a free-response question. Be prepared to answer multiple-choice questions on reading files.

In addition to the above `Scanner` constructor, you have to be familiar with the `Scanner` methods included in the *Java Quick Reference*: `nextInt`, `nextDouble`, `nextBoolean`, `nextLine`, `next`, `hasNext`, and `close`. Three of these methods, `nextInt`, `nextDouble`, and `nextBoolean`, throw an `InputMismatchException` if the next token in the file is in an unacceptable format or out of range.

↓ The `next` method returns the next token (a `String`) from the input file. If this is the last token on the line, `next` leaves the newline character in the stream. If the next call is to `nextLine`, another `nextLine` has to be called first to remove the newline character left in the stream. That call will return an empty string. All this may lead to unexpected results, so the Exam Development Committee excluded mixing `nextLine` calls and other `Scanner` method calls on the same input file.

↑

Once the file reading is finished, the file has to be closed by calling `Scanner`'s `close` method.

Using `Scanner` for reading keyboard input won't be tested.

**18**

Suppose a `Scanner` object is associated with an open text file. The file contains integer scores, either one per line or several on one line, separated by a space. This `Scanner` object is passed to the `getAverageScore` method, which returns the average of all the numbers in the file. Consider the following two versions of this method.

Version 1:

```
public double getAverageScore(Scanner input)
{
    int sum = 0;
    int count = 0;

    while (input.hasNext())
    {
        int score = input.nextInt();
        sum += score;
        count++;
    }
    return (double)sum / count;
}
```

Version 2:

```
public double getAverageScore(Scanner input)
{
    int sum = 0;
    int count = 0;

    while (input.hasNext())
    {
        String line = input.nextLine();
        for (String s : line.split(" "))
        {
            int score = Integer.parseInt(s);
            sum += score;
            count++;
        }
    }
    return (double)sum / count;
}
```

Which of the two versions will work as specified?

- (A) Neither works, because of the newline characters in the file
- (B) Only Version 1 works
- (C) Only Version 2 works
- (D) Both versions work as specified

➡ Scanner's `hasNext` method skips all “white space,” including spaces, tabs, and newline characters. The same is true for the `nextInt` method. So, Version 1 will work fine. The `nextLine` method in Version 2 reads the next line from the file, up to and including the next newline character, but it returns a string with newline stripped off. So, Version 2 works fine, too. The answer is D.

(If the numbers were separated by, say, commas, in addition to white space, Version 1 would throw a `NumberFormatException`, because `nextInt` expects only digits, possibly with a `+` or `-` sign in front.) ➡